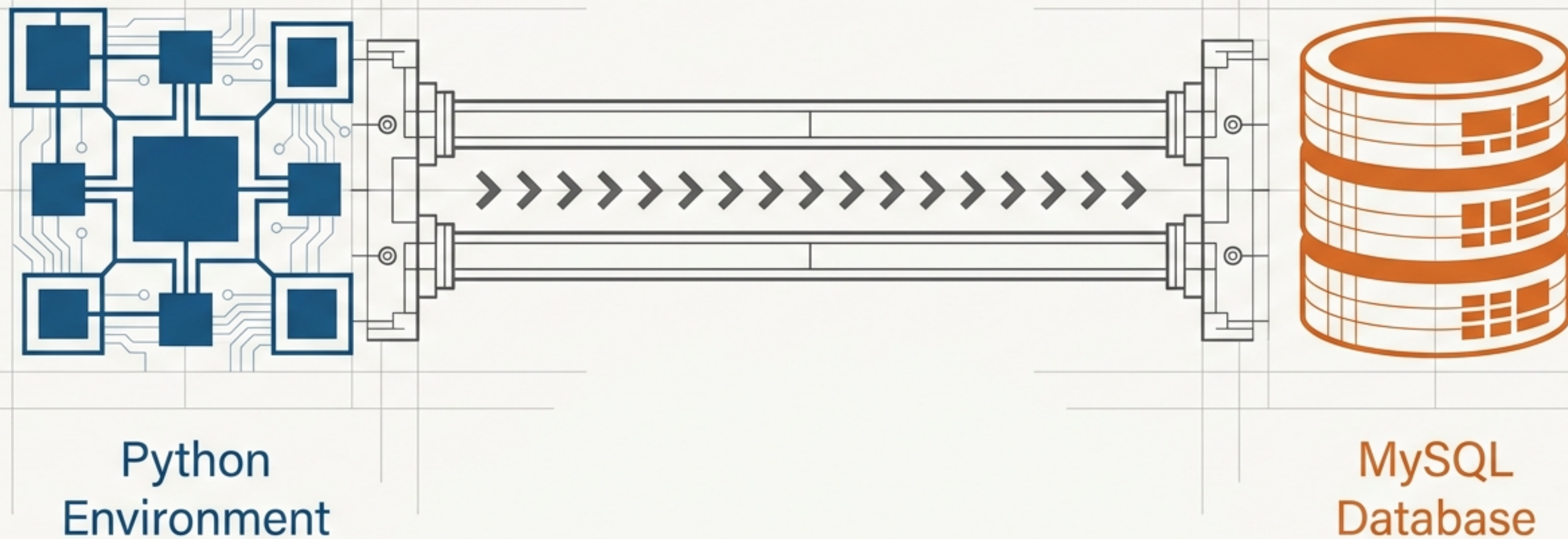
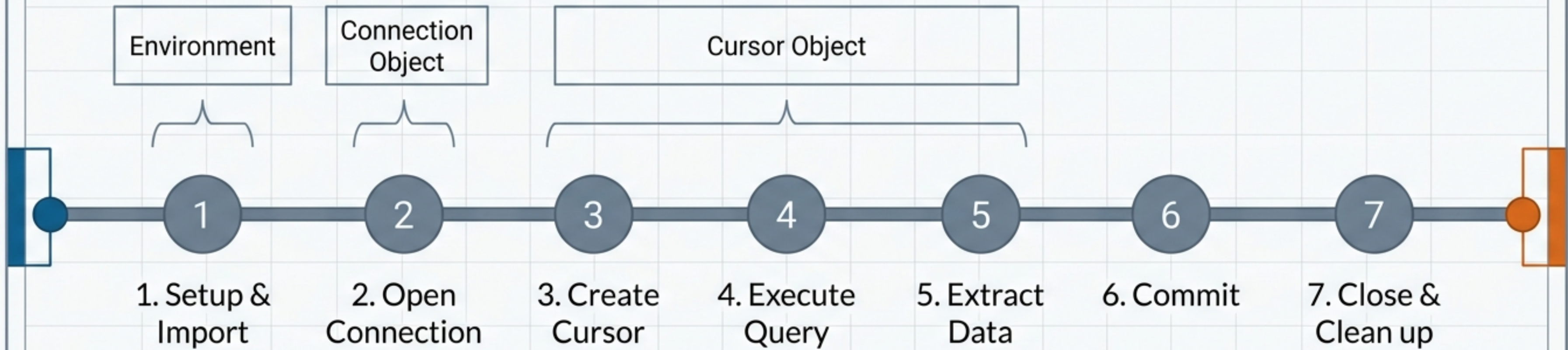


Interfacing Python with MySQL

The complete architectural guide to building database-driven Python applications.



The 7-Step Connectivity Pipeline



Phase 1: Preparing the Environment

System Command Line

```
> pip install mysql_connector_python
```

Python Script

```
import mysql.connector as m
```

We define `m` as an alias—any valid identifier works—to streamline and shorten function calls throughout the rest of the script.

Phase 2: Opening the Database Connection

```
con = m.connect(host='localhost',  
                user='root',  
                passwd='1234',  
                database='class')
```

host: Where the database is hosted (typically 'localhost').

user: The connecting user account (often 'root').

passwd: The specific password for the user account.

database: The exact database/schema targeted for data usage.



Verification Sidebar

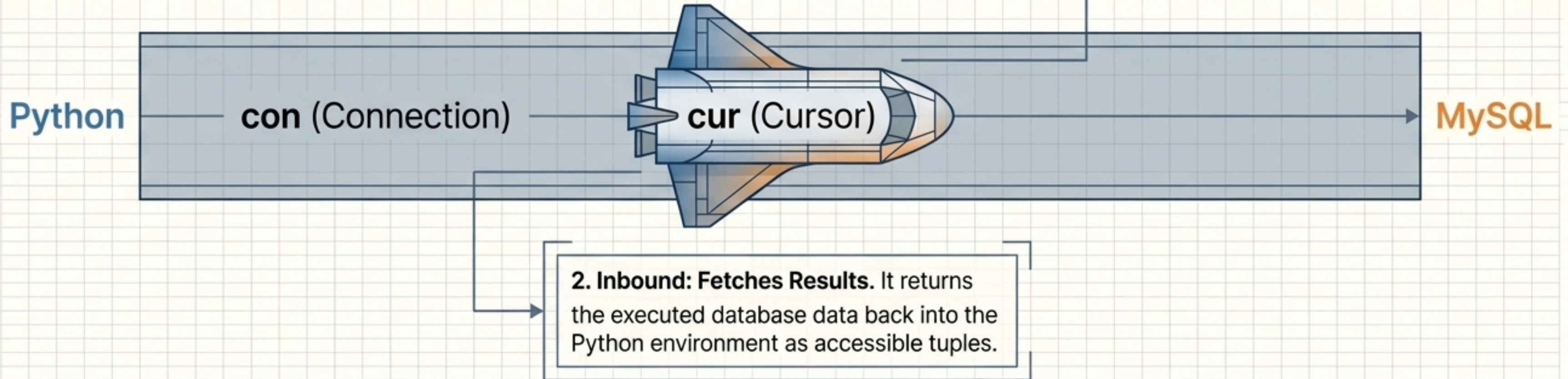
```
print(con.is_connected())
```

True ●

Phase 3: Deploying the Cursor Object

```
cur = con.cursor()
```

1. Outbound: Executes SQL Commands.
It carries raw SQL string queries from your Python code directly to the database engine.



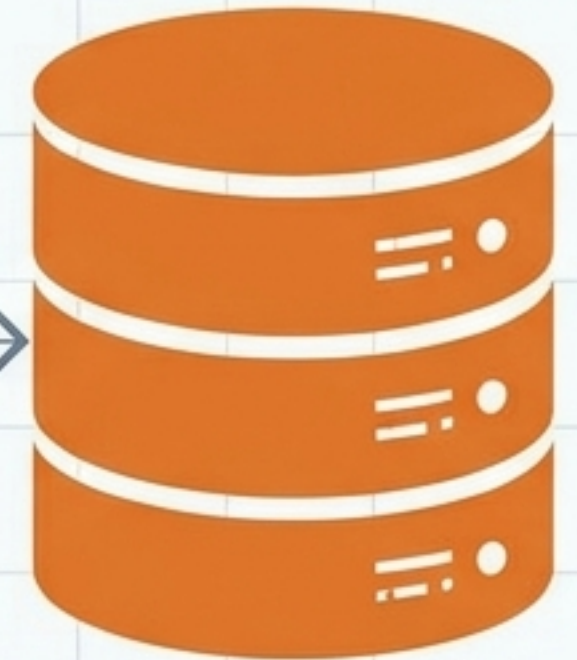
Phase 4: Executing Queries Across the Bridge

String Definition

```
query = "SELECT *  
FROM users"
```

Execution

```
cur.execute(query)
```



Key Insight Panel

To Python, a query is simply a text string. The `execute()` function pushes this string across the connection to MySQL. The database runs the command, and the resulting data is stored directly inside the `cur` object, waiting to be extracted.

Phase 5: Extracting the Data Payload

`cur.fetchall()`

Returns: List of Tuples (all retrieved records).

`cur.fetchone()`

Returns: Single Tuple (one record from the result set).

`cur.fetchmany(n)`

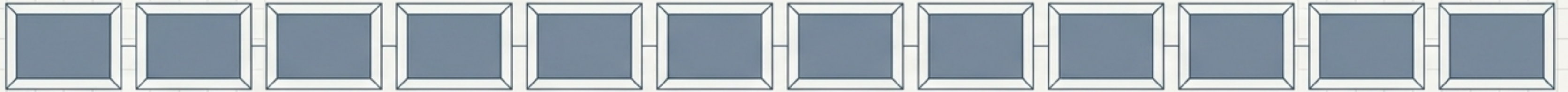
Returns: List of Tuples (n records as specified).

Implementation Sidebar

```
data = cur.fetchall()
for row in data:
    print(row)
```

Because rows arrive packed inside tuples, use a standard `for` loop to iterate through and display each individual record.

Monitoring State: The Rowcount Property



Total rows loaded into cursor memory: 10

```
cur.execute("select * from student")
```



All 10 rows loaded into cursor memory.

```
data = cur.fetchone()
```



cur.rowcount outputs 1

```
data = cur.fetchmany(4)
```



cur.rowcount outputs 5

```
data = cur.fetchone()
```



cur.rowcount outputs 6

```
data = cur.fetchall()
```



cur.rowcount outputs 10

Crucial Rule:

`cur.rowcount` returns the number of rows fetched from the cursor up to that exact moment, NOT the total number of records the executed query produced.

The Static Query Lexicon

DDL & Meta (Structure)	DML (Data Manipulation)	Schema Alteration (Modification)
<pre>cur.execute("CREATE TABLE FEES (ROLLNO INT, NAME VARCHAR(20), AMOUNT INT)")</pre>	<pre>cur.execute("SELECT * FROM STUDENT WHERE MARKS>90")</pre>	<pre>cur.execute("ALTER TABLE STUDENT ADD AGE INT")</pre>
<pre>cur.execute("DROP TABLE STUDENT")</pre>	<pre>cur.execute("UPDATE STUDENT SET MARKS=100 WHERE MARKS=40")</pre>	<pre>cur.execute("ALTER TABLE DROP AGE")</pre>
<pre>cur.execute("SHOW TABLES")</pre>	<pre>cur.execute("DELETE FROM STUDENT WHERE MARKS<50")</pre>	<pre>cur.execute("ALTER TABLE STUDENT MODIFY GRADE CHAR(3)")</pre>
<pre>cur.execute("DESC STUDENT")</pre>		

Dynamic Interactions: Parameterized Queries

The .format() Method

```
"select * from student where rollno  
= {}".format(rn)
```

```
"select * from student where  
{ } = {}".format(col_name, col_value)
```

```
"select * from student where  
class = { } and section = '{ }'".  
format(cls, sec)
```

Modern f-strings

```
f"select * from student where  
rollno = {rn}"
```

```
f"select * from student where  
{col_name} = {col_value}"
```

```
f"select * from student where  
class = {cls} and section = '{sec}'"
```

Both methods construct a finalized static string before pushing it across the bridge to MySQL.

Phase 6 & 7: Transaction and Cleanup

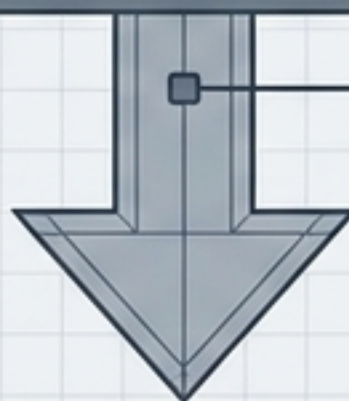


Action Block 1

The Stamp of Approval

```
con.commit()
```

Data changes aren't permanent automatically. After executing INSERT, UPDATE, or DELETE queries, you must **commit the transaction** on the **connection** object to save the changes permanently to the database.



Action Block 2

Closing the Gate

```
con.close()
```

Databases support a limited number of simultaneous connections. **Freeing up** resources by **explicitly closing the connection object** is a mandatory cleanup practice.

The Blueprint Complete



```
# 1. Setup
import mysql.connector as m

# 2. Connect
con = m.connect(host='localhost', user='root', passwd='sys', database='student')

# 3. Cursor
cur = con.cursor()

# 4. Execute
cur.execute("UPDATE STUDENT SET MARKS=100 WHERE MARKS=40")

# 5. Extract (n/a for update, but typically fetchall here)
print(cur.rowcount, "RECORD UPDATED")

# 6. Commit
con.commit()

# 7. Cleanup
con.close()
```

The Python-MySQL Bridge is complete. Ready for production.